

Lesson 1 — Closer To The Metal

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

Compile, then run. Two steps.

```
clang++ -std=c++17 -Wall file.cpp -o file
./file
```

Part	Means
clang++	the compiler (g++ works the same)
-std=c++17	the 2017 language version
-Wall	warn me about everything
-o file	what to name the result
./file	run it, from this folder

Change the file → compile again. Running without recompiling runs the OLD program.

The smallest program

```
#include <iostream>

int main() {
    std::cout << "Hello\n";
    return 0;
}
```

- `#include` — paste in another file, before compiling
- `int main()` — where every program starts
- `std::` — the standard library's namespace
- `;` — ends every statement
- `return 0;` — finished with no problem

Types

```
int    score    = 0;        // whole number
double accuracy = 0.0;     // decimal
bool   playing  = true;     // true / false
char   grade    = 'A';     // ONE character
std::string name = "";      // text
```

Always give a starting value. An uninitialised variable holds junk, not zero.

Whole-number division

```
7 / 2          // 3, not 3.5 - it TRUNCATES
7.0 / 2        // 3.5
7 % 2          // 1 (the remainder)
```

One side must be a decimal, or the fraction is thrown away.

Random, done properly

```
std::random_device seed;
std::mt19937 generator(seed());
std::uniform_int_distribution<int> die(1, 6);

int roll = die(generator);
```

`rand() % 6 + 1` works and is slightly **biased**.

The loop

```
while (playing) {
    show();                // RENDER
    std::cin >> cmd;       // INPUT (this
    WAITS)
    handle(cmd);           // UPDATE
}
```

Section A — Recall

A1. What does a compiler do? Name two things you get in return for having to compile. [3 marks]

A2. Write the full command to compile `game.cpp` into a program called `game`, with all warnings on. [2 marks]

A3. What does `-Wall` do, and why should you always use it? [2 marks]

A4. A game runs at 60 frames per second. How many milliseconds does one frame get? Show your working, and say what has to happen in that time. [3 marks]

A5. Why must you declare the type of every variable in C++, when Python does not? [2 marks]

Section B — Predict the output

Write your answer down before compiling anything.

B1. What does each line print? [4 marks]

```
std::cout << 7 / 2 << "\n";
std::cout << 7.0 / 2 << "\n";
std::cout << 7 % 2 << "\n";
std::cout << 1000 / 60 << "\n";
```

B2. The player's score is 87 out of 100. What does this print, and why is it almost certainly not what the author wanted? [3 marks]

```
int score = 87;
double accuracy = score / 100;
std::cout << accuracy;
```

B3. What is printed?

[3 marks]

```
bool playing = true;
std::cout << playing << "\n";
std::cout << (3 > 5) << "\n";
```

B4. A student types `banana` when asked for a number. What is in `guess` afterwards, and what does the program print? [4 marks]

```
int guess = 0;
std::cout << "Pick a number: ";
std::cin >> guess;
std::cout << "You picked " << guess << "\n";
```

Section C — Find and fix the bug**C1. This will not compile. There are **two** mistakes. Name both.**

[3 marks]

```
int main() {
    std::cout << "Hello"
    return 0;
}
```

C2. A student insists their change is not working. The code is correct. What have they almost certainly forgotten to do? [3 marks]**C3. This compiles and prints a different nonsense number every time it is run. Why?**

[4 marks]

```
int main() {
    int total;
    for (int i = 1; i <= 10; i++) {
        total = total + i;
    }
    std::cout << total << "\n";
}
```

Section D — Build it

- ☐ **Checkpoint 1.** Compile and run a hello-world program. **Do not move on until this works.**
- ☐ **Checkpoint 2.** Ask the player their name and read it with `getline`. Ask for a number and read it with `>>`. Print both back.
- ☐ **Checkpoint 3.** Roll one die with `uniform_int_distribution` and print it. Run the program five times and check you get different numbers.
- ☐ **Checkpoint 4.** Build a `while` loop that keeps playing rounds until the player types `quit`. Keep a score in a variable outside the loop.
- ☐ **Checkpoint 5.** Make it a duel: you and an opponent each roll two dice, higher total wins the round, first to three wins the game.
- ☐ **Checkpoint 6.** Add a `stats` command that rolls 100,000 dice and prints how often each face came up as a percentage. Are they even?

THE TWO-STEP HABIT

Every single time you change the file: **compile, then run.** If your change seems to

have done nothing, you are almost certainly running the old program. This happens to everybody, this week and forever.</p>

Section E — Design challenge

NO RIGHT ANSWER

E1. The error message game. Break your program in five different ways, one at a time, and write down the message you get. Which were helpful? Which pointed at the wrong line, and why?

E2. Python needs no compile step. C++ does. What does each one buy you? Think about *writing* a game, and then about *shipping* one to a player who does not have Python installed.

E3. If C++ is so fast, why are almost all game *tools* — level editors, build scripts, asset pipelines — usually written in Python or C#? What is different about those jobs?

Stretch goals

1. Best of five instead of best of three.
2. Let the player choose the number of sides. What if they pick 1? Or 0? Or -4? Handle it.
3. Compare `uniform_int_distribution` with `rand() % 6 + 1` over a million rolls. Can you see the bias?
4. Time a million rolls with `<chrono>`, then do the same in Python. The ratio is the answer to "why C++?".
5. Make the goblin cheat, but only when losing. Is the game more fun? Should the player be able to tell?